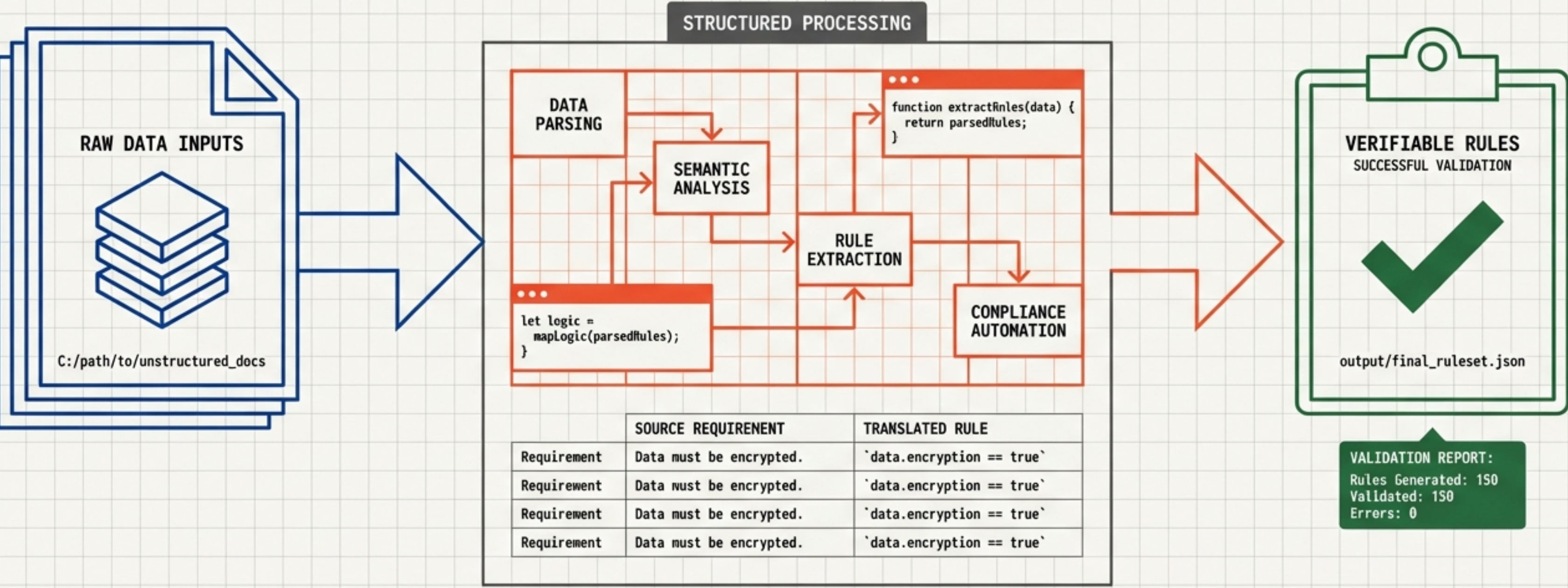


# Project FFEELAA

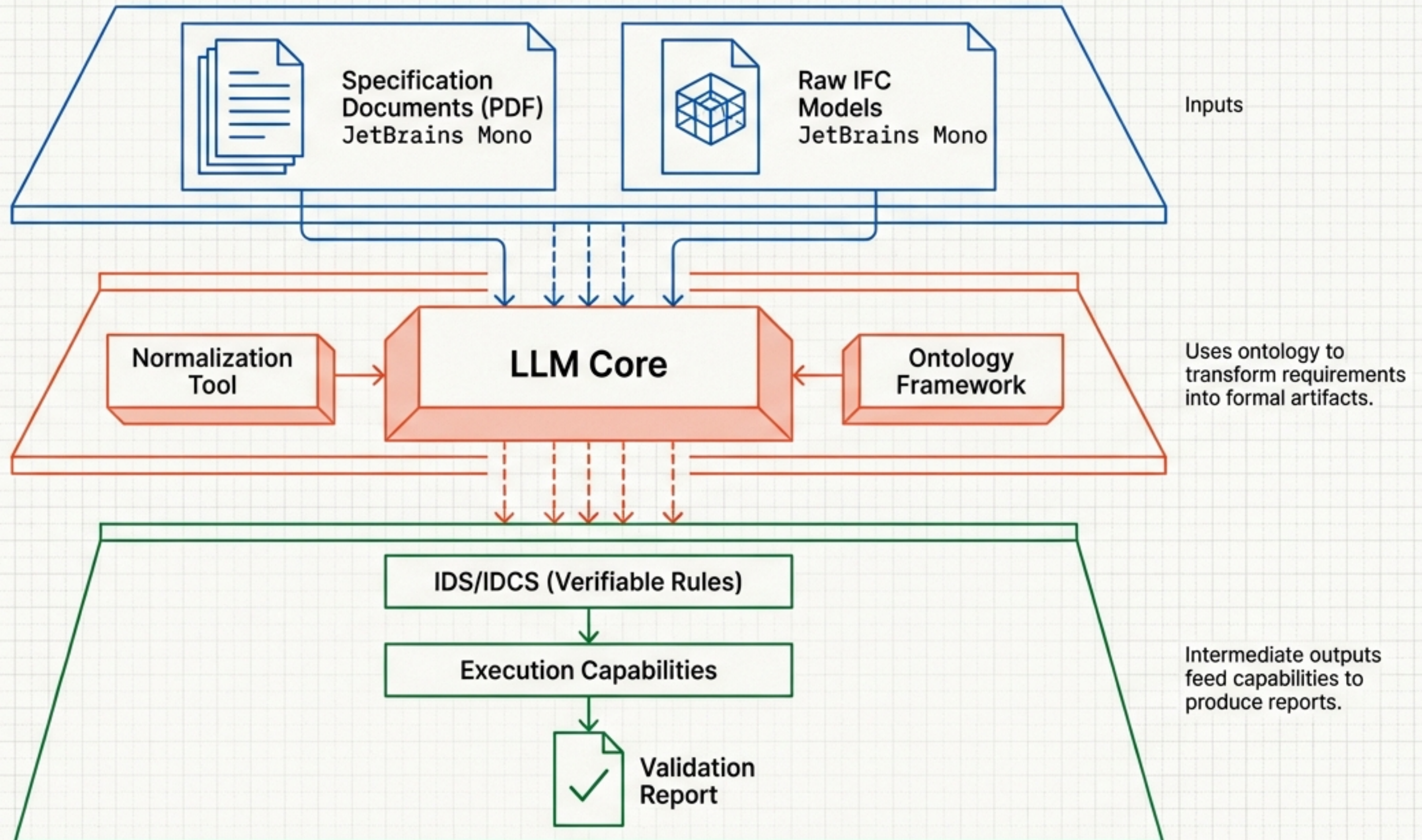
Automated compliance from unstructured requirements to verifiable rules.

Technical Onboarding and System Architecture Reference



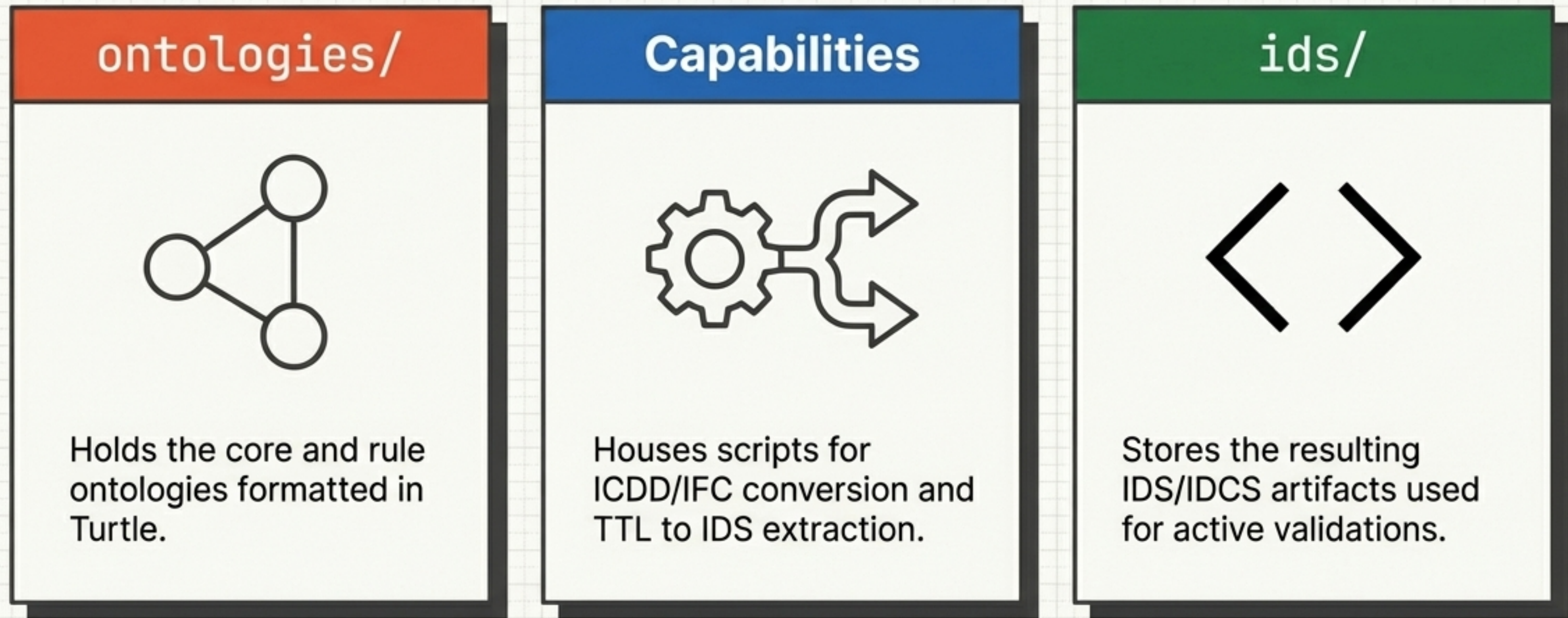


# Unstructured text becomes executable logic





# The project architecture is divided into logic, execution, and artifacts





# Capability breakdown: Normalization vs. Extraction

|                  |  <code>convert_icdd_ontology.py</code> |  <code>ttl_to_ids.py</code> |
|------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Primary Function | Normalizes IFC & applies ICDD mappings                                                                                  | Filters IDS-expressible rules from Turtle.                                                                     |
| Library Stack    | ICDD processing tools                                                                                                   | RDFLib parser                                                                                                  |
| Core Output      | Normalized IFC                                                                                                          | XML-based specifications                                                                                       |

Directory: `capability/package/capability/plugin/capability/`



# Dynamic matching prevents rigid namespace coupling



The TTL parser (using RDFLib) avoids hardcoding namespaces. Instead, it dynamically searches for predicates matching the localname. This ensures flexible rule extraction.



# Execution Phase 1: Containerize and Normalize

**Step 1:** Build Example  
ICDD Container



**Step 2:** Normalize IFC  
& Apply Mappings



**Result:**

IFC\_files/TIP01-ARQ-  
MOD\_R03.norm.ifc

Assumes virtual environment is fully  
configured prior to execution.



# Execution Phase 2: Transformation and Validation

Validation is an optional step, executed via the ifctester module.

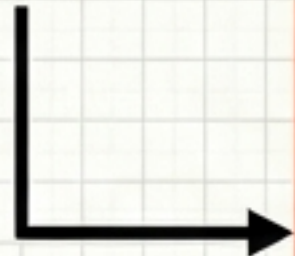
**Step 3:** Transform TTL  
Ontologies into IDS



**Step 4:**  
Validate IDS



**Node 3**



```
ifctester.ids (Ids.to_xml(...))
```

**Node 4**



Validation is an optional  
step, executed via the  
ifctester module.



# Verifying the TTL rule extractor with pytest

Dedicated unit tests ensure the TTL to IDS transformation accurately filters expressible rules before entering the main validation pipeline.

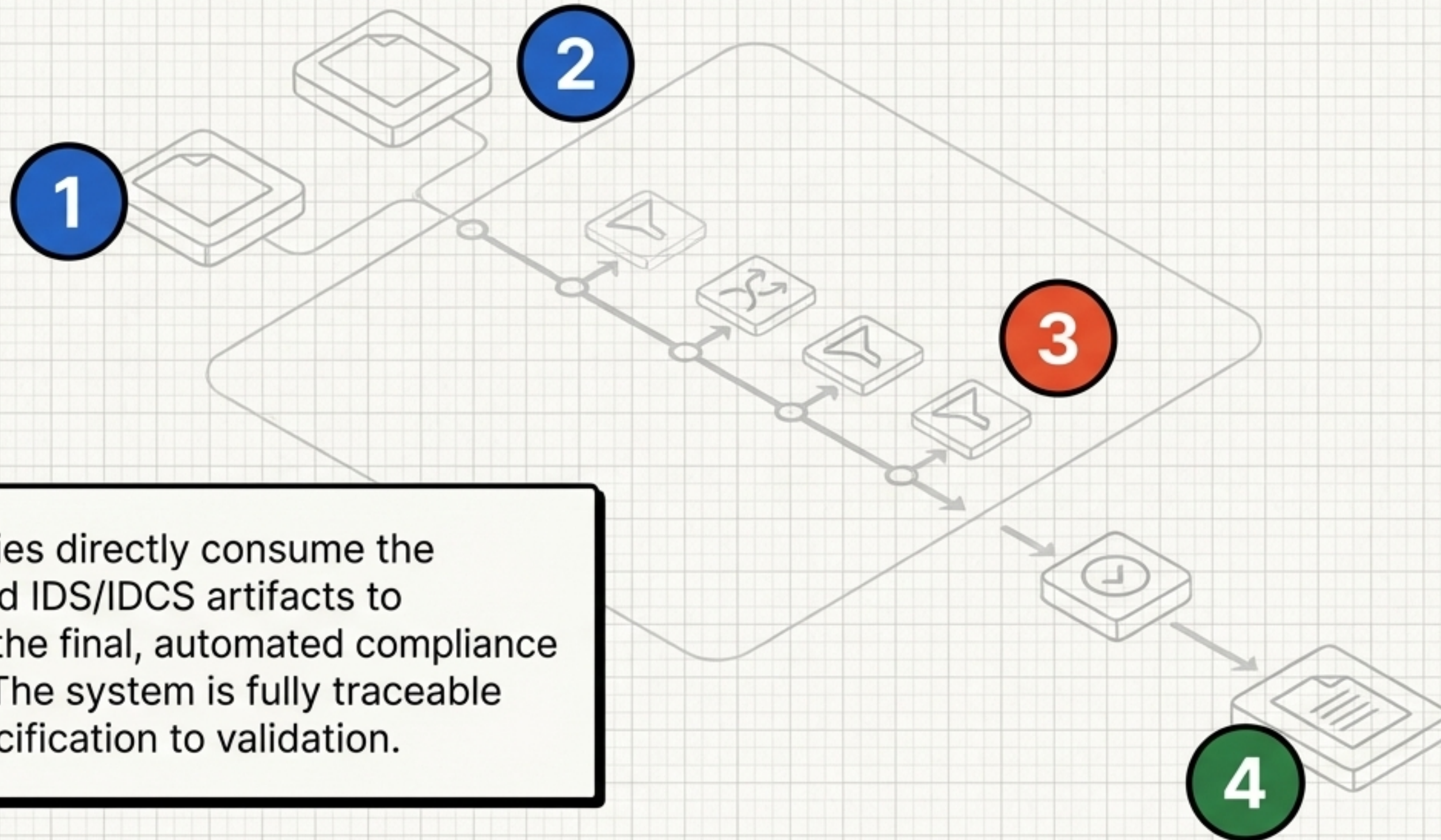
```
tests/test_ttl_to_ids_read_fela_computable_rules.py
```

**test\_extract\_rules** **PASSED**

This dedicated unit test verifies that the TTL to IDS transformation process correctly identifies and extracts only the expressible rules from the input ontologies, ensuring data integrity before the main validation pipeline begins.



# The refinery in motion: from raw PDF to verified report



Capabilities directly consume the generated IDS/IDCS artifacts to produce the final, automated compliance reports. The system is fully traceable from specification to validation.